

Project_1_8420

March 7, 2025

1 DSA 8420 Spring 2025 Project 1

1.0.1 Due March 11, 2025

This project consists of two problems. In the first problem, we solve and analyze the agricultural problem presented in Homework 2. In the second problem, we examine a linear system with conflicting equations and compute a measure of its inconsistency.

```
[1]: # packages
import pyomo.environ as pyo
from pyomo.opt import SolverFactory
```

1.1 Q1: The Agricultural Problem

Problem description: A farmer owns a farm that produces corn, soybean, and oats. There are 12 acres of land available for cultivation. Each crop that is planted has certain requirements for labor and capital. These data along with the net profit figure are given in the accompanying table.

	<i>Labor (hr)</i>	<i>Capital (\$)</i>	<i>Net profit (\$)</i>
Corn (per acre)	6	36	40
Soybeans (per acre)	6	24	30
Oats (per acre)	2	18	20

The farmer has \$360 available for capital and knows that there are 48 hr available for working these crops. How much of each crop should be planted to maximize profit?

Modeling: Let x_1 , x_2 , x_3 represent the number of acres we use to plant corn, soybeans, and oats, respectively. The problem can be modeled as a linear program:

$$\begin{array}{ll} \max & 40x_1 + 30x_2 + 20x_3 & (\text{profit}) \\ \text{s.t.} & x_1 + x_2 + x_3 \leq 12 & (\text{land}) \\ & 6x_1 + 6x_2 + 2x_3 \leq 48 & (\text{labor}) \\ & 36x_1 + 24x_2 + 18x_3 \leq 360 & (\text{capital}) \\ & x_1, x_2, x_3 \geq 0 \end{array}$$

Tasks:

1. (18 points) With the accompanying data file 'agriculture.dat', develop an abstract model using Pyomo to solve the problem. Your code should take data from 'agriculture.dat' and print to screen the optimal solution and its corresponding optimal value. (Hint: You can use the Jupyter Notebook file in Homework 7 as a template.)

```
[2]: # Create an Abstract model
model = pyo.AbstractModel()
```

```
[3]: # Build the index set
model.Crop = pyo.Set()
```

```
[4]: # build params
# model.w_land = pyo.Param(model.Crop, within=pyo.NonNegativeReals) # not
# included
model.w_labor = pyo.Param(model.Crop, within=pyo.NonNegativeReals)
model.w_capital = pyo.Param(model.Crop, within=pyo.NonNegativeReals)
model.profit = pyo.Param(model.Crop, within=pyo.NonNegativeReals)

model.avail_land = pyo.Param(within=pyo.NonNegativeReals)
model.avail_labor = pyo.Param(within=pyo.NonNegativeReals)
model.avail_capital = pyo.Param(within=pyo.NonNegativeReals)

# build decision vars
model.x = pyo.Var(model.Crop, within=pyo.NonNegativeReals)
```

```
[5]: # Build objective function
def obj_value_rule(model):
    return sum(model.profit[i] * model.x[i] for i in model.Crop)

model.obj = pyo.Objective(rule = obj_value_rule, sense = pyo.maximize)
```

```
[6]: # constraints
def land_rule(model):
    return sum(model.x[i] for i in model.Crop) <= model.avail_land

def labor_rule(model):
    return sum(model.w_labor[i] * model.x[i] for i in model.Crop) <= model.
    avail_labor

def capital_rule(model):
    return sum(model.w_capital[i] * model.x[i] for i in model.Crop) <= model.
    avail_capital

model.land = pyo.Constraint(rule = land_rule)
model.labor = pyo.Constraint(rule = labor_rule)
model.capital = pyo.Constraint(rule = capital_rule)
```

```
[7]: # Load data (original agriculture.dat file)
instance = model.create_instance('agriculture.dat')
solver = SolverFactory('glpk')
results = solver.solve(instance)

# Print the optimization results
instance.display() # List of all optimization results
```

Model unknown

Variables:

```
x : Size=3, Index=Crop
   Key      : Lower : Value : Upper : Fixed : Stale : Domain
   Corn :    0 :   6.0 :   None : False : False : NonNegativeReals
   Oats :    0 :   6.0 :   None : False : False : NonNegativeReals
   Soybeans : 0 :   0.0 :   None : False : False : NonNegativeReals
```

Objectives:

```
obj : Size=1, Index=None, Active=True
     Key : Active : Value
     None : True : 360.0
```

Constraints:

```
land : Size=1
      Key : Lower : Body : Upper
      None : None : 12.0 : 12.0
labor : Size=1
      Key : Lower : Body : Upper
      None : None : 48.0 : 48.0
capital : Size=1
        Key : Lower : Body : Upper
        None : None : 324.0 : 360.0
```

```
[8]: # optimal sol
print("Optimal Solution:")
for i in instance.Crop:
    print(i, pyo.value(instance.x[i]))

# max profits
print('\nMax Profit: $', pyo.value(instance.obj)) # optimal objective val
```

Optimal Solution:

Corn 6.0

Soybeans 0.0

Oats 6.0

Max Profit: \$ 360.0

2. Adjust the parameters (avail_land, avail_labor, avail_capital) in 'agriculture.dat' slightly

away from the current value of (12, 48, 360) and re-run the code to answer the following three (independent) sets of questions.

Set 1:

- a. (2 points) Modify the value of 'avail_land' to complete the second row of the table below.

avail_labor = 48, avail_capital = 360

avail_land (acre)	9	10	11	12	13	14	15
maximum profit (\$)							

```
[9]: # Load modified data
instance = model.create_instance('ag_set1a.dat') # current avail_land == 15
solver = SolverFactory('glpk')
results = solver.solve(instance)
print('Optimal value: ', pyo.value(instance.obj))
```

Optimal value: 390.0

Results:

avail_labor = 48, avail_capital = 360

avail_land (acre)	9	10	11	12	13	14	15
maximum profit (\$)	330	340	350	360	370	380	390

- b. (2 points) Around the current value of the parameters, what is the rate of change of the profit with respect to the acres of land?
- Rate of change of profit is \$10 for each 1 acre increment of avail_land.
- c. (2 points) If one acre of land is available for rent during the period of cultivation, what is the highest price the farmer would like to pay? (The price is called the shadow price of the land.)
- The farmer can make at most a maximum profit of \$10 with each additional 1 acre of avail_land, thus the highest price the farmer should pay should be any value lower than \$10.

Set 2:

- d. (2 points) Modify the value of 'avail_labor' to complete the second row of the table below.

avail_land = 12, avail_capital = 360

avail_labor (hr)	45	46	47	48	49	50	51
maximum profit (\$)							

```
[10]: # Load modified data
instance = model.create_instance('ag_set2a.dat') # cur avail_labor == 51
solver = SolverFactory('glpk')
results = solver.solve(instance)
print('Optimal value: ', pyo.value(instance.obj))
```

Optimal value: 375.0

Results:

avail_land = 12, avail_capital = 360

avail_labor (hr)	45	46	47	48	49	50	51
maximum profit (\$)	345	350	355	360	365	370	375

- e. (2 points) Around the current value of the parameters, what is the rate of change of the profit with respect to the hours of labor?
 - Rate of change for profit is \$5 for each 1 laborer increment of avail_labor.
- f. (2 points) If the farmer wants to hire a helper with the same efficiency, how much at most would the farmer pay the helper for an hour of work? (The rate is the shadow price of the labor.)
- As the farmer makes an additional max profit of \$5 an hour for each additional laborer he hires, he should at most pay the laborer any value less than \$5 an hour to be profitable.

Set 3:

- g. (2 points) Modify the value of 'avail_capital' to complete the second row of the table below.

avail_land = 12, avail_labor = 48

avail_capital (\$)	330	340	350	360	370	380	390
maximum profit (\$)							

```
[11]: # Load modified data
instance = model.create_instance('ag_set3a.dat') # cur avail_capital == 390
solver = SolverFactory('glpk')
results = solver.solve(instance)
print('Optimal value: ', pyo.value(instance.obj))
```

Optimal value: 360.0

- Maximum profit is \$360 (static) for any avail_capital between \$330 and \$390.
- h. (2 points) Around the current value of the parameters, what is the rate of change of the profit with respect to the dollars of capital?
 - Rate of change is \$0.

- i. (2 points) If a bank offers to lend the farmer one dollar to expand his capital investment (hypothetically, don't laugh), what is the highest acceptable interest rate for the farmer? (The rate is the shadow price of the capital.)
- The highest acceptable interest rate for the farmer is 0%; borrowing an additional \$ would not generate any additional profit, making it unprofitable to take the loan at a positive interest rate.

1.2 Q2: Inconsistency of A Linear System

Problem description: Consider the following system of linear equations:

$$2x_1 + 3x_2 = 6$$

$$2x_1 + 3x_2 = 8$$

$$x_1 + x_2 = 4$$

Clearly, this system is inconsistent – there is no pair (x_1, x_2) that simultaneously satisfies all three equations. Given any (x_1, x_2) , the absolute error for the first equation is defined as $|2x_1 + 3x_2 - 6|$. Similarly, the absolute error for the second and third equation is defined as $|2x_1 + 3x_2 - 8|$ and $|x_1 + x_2 - 4|$, respectively.

Modeling: We measure the inconsistency of the linear system by finding the minimum possible value of the maximum absolute error across all three equations. In mathematical terms, we seek to solve the following optimization problem:

$$\min_{x_1, x_2} \max\{|2x_1 + 3x_2 - 6|, |2x_1 + 3x_2 - 8|, |x_1 + x_2 - 4|\}$$

Tasks:

3. (6 points) Reformulate the above optimization problem as a linear program.

Min z

s.t.:

- $z \geq |2x_1 + 3x_2 - 6|$
- $z \geq |2x_1 + 3x_2 - 8|$
- $z \geq |x_1 + x_2 - 4|$

(converting absolute values to linear constraints)

Min z

s.t.:

- $z \geq 2x_1 + 3x_2 - 6$
- $z \geq -2x_1 - 3x_2 + 6$

- $z \geq 2x_1 + 3x_2 - 8$
- $z \geq -2x_1 - 3x_2 + 8$
- $z \geq x_1 + x_2 - 4$
- $z \geq -x_1 - x_2 + 4$

4. (18 points) Build a concrete model using Pyomo to solve the linear program. Your code should display the optimal solution and its corresponding optimal value.

```
[12]: # concrete model
model = pyo.ConcreteModel()

# vars
model.x1 = pyo.Var(within=pyo.Reals)
model.x2 = pyo.Var(within=pyo.Reals)
model.z = pyo.Var(within=pyo.Reals)

[13]: # objective: min z
model.obj = pyo.Objective(expr=model.z, sense=pyo.minimize)

[14]: # constraints
def constraint_1_rule(model):
    return model.z >= 2 * model.x1 + 3 * model.x2 - 6

def constraint_2_rule(model):
    return model.z >= -2 * model.x1 - 3 * model.x2 + 6

def constraint_3_rule(model):
    return model.z >= 2 * model.x1 + 3 * model.x2 - 8

def constraint_4_rule(model):
    return model.z >= -2 * model.x1 - 3 * model.x2 + 8

def constraint_5_rule(model):
    return model.z >= model.x1 + model.x2 - 4

def constraint_6_rule(model):
    return model.z >= -model.x1 - model.x2 + 4

model.constraint_1 = pyo.Constraint(rule=constraint_1_rule)
model.constraint_2 = pyo.Constraint(rule=constraint_2_rule)
model.constraint_3 = pyo.Constraint(rule=constraint_3_rule)
model.constraint_4 = pyo.Constraint(rule=constraint_4_rule)
model.constraint_5 = pyo.Constraint(rule=constraint_5_rule)
model.constraint_6 = pyo.Constraint(rule=constraint_6_rule)
```

```
[15]: # results
      solver = pyo.SolverFactory('glpk')
      result = solver.solve(model)

      print(f"Optimal x1: {model.x1.value}")
      print(f"Optimal x2: {model.x2.value}")
      print(f"Optimal z(minimized max absolute deviation): {model.z.value}")
```

Optimal x1: 2.0

Optimal x2: 1.0

Optimal z(minimized max absolute deviation): 1.0

```
[ ]: !jupyter nbconvert --to pdf Project_1_8420.ipynb
```

```
[ ]:
```